1. *Unqualified name*: This is basically the name of a class, function or a constant, devoid of counting a reference to any namespace. Also, if the user is new to namespacing, then this is the basic category you need to work from.
   For example:

```
<?php
Namespace My_assignment;
Class Myclass {
            Static function staticMethod()
{
                        Echo 'Hi, Everyone!';
            }
}
Myclass:staticMethod();
```

2. *Qualified name:* This is the case in which we know how to access the sub-namespace chain of command along with using the backslash notation.
   For example:

```
<?php
Namespace My_assignment;
Require 'my_assignment/database/connection.php';
$connection = new database\connection();
```

3. *Fully qualified name*: The above discussion on unqualified and qualified names is based on the namespace you are presently in. Also, these two categories can only be used to access definitions at that particular stage or to jump deeper into the namespace chain of command. But, if you need to access a function, a class or a constant that is residing at a superior stage in the chain of commands, then you need to employ the fully qualified name, which is the complete path. Your call needs to be made with a backslash notation. With this, PHP gets to know that this particular call should be determined from the global space as a replacement for approaching it moderately.
   For example:

```
<?php
Namespace My_assignment\database;
Require 'My_assignment/fileaccess/input.php;
$input = new\My_assignment\Fileaccess\Input();
```

Dynamic calls

PHP is a dynamic encoding language, so this functionality can be used for calling namespaced code. It is actually the same as instantiating variable classes or counting variable files. Also, never forget to remove the backslash at the time of storing a namespace name in a string.

Namespace keyword

A namespace keyword is not only the one that is used to describe a namespace, but it can also be used to explicitly determine the current namespace.

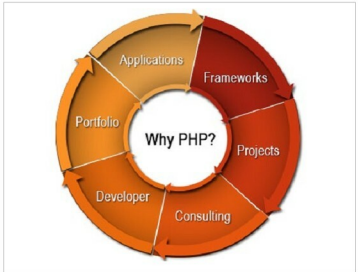

Figure 3: Scope of PHP

Namespace constant

As this keyword does not determine the current class name, hence the namespace keyword cannot be used to determine the current namespace. This is why we use the namespace constant.

Aliasing/importing

In PHP, namespacing supports importing. This is basically referred to as aliasing or importing, which can only be done on classes, interfaces and namespaces. Importing, which is the most useful feature of namespacing, allows the easy use of external code packages such as libraries without the worry of contradictory names. Importing is thus attained by the use of a keyword.

PHP is the perfect choice for you because of the following reasons.

1. *Quick loading time:* It has the fastest speed in loading a site. This is because it runs in its own memory space.
2. *Low cost software:* Since PHP is an open source language, most of the tools connected with the code don't need to be paid for.
3. *Low cost hosting:* PHP can easily be executed on a Linux server, which is simply available via a hosting provider without any additional cost.
4. *Flexibility of the database:* Database connectivity can be easily done in PHP.

These are some of the common reasons why PHP is considered the best programming language. END

By: Meghraj Singh Beniwal

The author has a B.Tech in electronics and communication, is a freelance writer and an Android app developer. He currently works as an automation engineer at Infosys, Pune. He can be contacted at *meghrajsingh01@rediffmail.com* or *meghrajwithandroid@gmail.com*.